C PROGRAMMING

Lesson 11 — Functions

Based on: Bro Code C Tutorial Series

1. What is a Function?

A function is a reusable block of code that performs a specific task. Instead of writing the same code over and over, you write it once inside a function and call it whenever you need it.

```
// Without functions -- repetitive and messy
printf("Happy Birthday to you!\n");
printf("Happy Birthday to you!\n");
printf("Happy Birthday dear Bro!\n");
printf("Happy Birthday to you!\n");

printf("Happy Birthday to you!\n");  // copied again...
printf("Happy Birthday to you!\n");
printf("Happy Birthday dear Bro!\n");
printf("Happy Birthday to you!\n");
```

2. Defining and Calling a Function

A function has two parts: the definition (where you write what it does) and the call (where you use it).

```
#include <stdio.h>

// Function DEFINITION -- written above main()
void happyBirthday() {
    printf("Happy Birthday to you!\n");
    printf("Happy Birthday to you!\n");
    printf("Happy Birthday dear Bro!\n");
    printf("Happy Birthday to you!\n");
}

int main() {
    happyBirthday();   // Function CALL -- runs the code above
    happyBirthday();   // call it again
    happyBirthday();   // and again
    return 0;
}
```

- **void** means the function returns nothing (just does something)
- **The function name** is followed by () — always include parentheses when calling

- **Define functions ABOVE main()** so the compiler knows they exist before they are called

> **Important:** Writing the function name without () does nothing. You must include the parentheses to actually call and run it.

3. Function Syntax Breakdown

```
return_type  function_name ( parameters ) {
    // function body
}

// Example:
void         happyBirthday (              ) {
    printf("Happy Birthday!\n");
}
```

Part	Meaning	Example
Return type	What the function gives back. void = nothing	`void`
Function name	Descriptive name for what it does	`happyBirthday`
Parameters	Inputs the function accepts (can be empty)	`() or (char name[])`
Function body	The code that runs when function is called	`{ printf(...); }`

4. Arguments and Parameters

To make a function work with different data, you pass values into it. The value you pass is called an argument. The variable inside the function that receives it is called a parameter.

```
// Function with parameters
void happyBirthday(char name[], int age) {
    printf("Happy Birthday dear %s!\n", name);
    printf("You are %d years old!\n", age);
}

int main() {
    char name[] = "Bro";
    int age = 10;

    happyBirthday(name, age);   // passing arguments
    return 0;
}
```

> **Argument vs Parameter:** "name" and "age" in main() are ARGUMENTS (the actual values you pass). "char name[]" and "int age" in the function definition are PARAMETERS (the variables that receive them).

5. Practice Problem

Practice: Student Report Card

Write a C program that uses TWO functions to print a student report card.

Function 1 — printHeader(char name[], int grade):

- Prints the student name and grade level as a header

Function 2 — printSubject(char subject[], float score):

- Prints a subject name and score formatted to 2 decimal places
- Also prints "Pass" if score >= 50, or "Fail" if below 50

In main(), ask the user for their name and grade, then call printHeader(). Then call printSubject() three times for three different subjects.

Expected Output (example)

```
===== Report Card =====
Student: Alex
Grade: 10
======================
Math        : 85.00  -- Pass
Science     : 47.50  -- Fail
English     : 91.00  -- Pass
```